01 LC 200 | Number of Islands (Medium)

Problem: Given an m x n 2D binary grid grid which represents a map of '1's (land) and '0's (water), return the number of islands. An island is surrounded by water and is formed by connecting adjacent lands horizontally or vertically. You may assume all four edges of the grid are all surrounded by water.

Constraints: m == grid.length; n == grid[i].length; 1 <= m, n <= 300; grid[i][j] is '0' or '1'.

```
class Solution {
public:
  int numIslands(vector<vector<char>>& grid) {
    int m = grid.size();
    int n = grid[0].size();
    int ans = 0;
    int dirs[5] = {-1, 0, 1, 0, -1};
    for (int i = 0; i < m; ++i) {
      for (int j = 0; j < n; ++j) {
        if (grid[i][j] != '1') {
          continue;
        }
        vector<pair<int, int>> stk{{i, j}};
        grid[i][j] = '0';
        while (!stk.empty()) {
          auto [a, b] = stk.back();
          stk.pop_back();
          for (int k = 0; k < 4; ++k) {
            int x = a + dirs[k], y = b + dirs[k +
              1];
            if (x >= 0 && x < m && y >= 0 && y < n
              && grid[x][y] == '1') {
              grid[x][y] = '0';
              stk.emplace_back(x, y);
            }
          }
        }
        ++ans;
      }
    }
    return ans;
  }
};
```

02 LC 133 | Clone Graph (Medium)

Problem: Return a deep copy of the connected undirected graph reachable from the supplied node. Each node has a value and neighbor list. Clone every node/link; return null for an empty graph.

Constraints: The number of nodes in the graph is in the range [0, 100].; 1 <= Node.val <= 100; Node.val is unique for each node.; There are no repeated edges and no self-loops in the graph.; The Graph is connected and all nodes can be visited starting from the given node.

```
class Solution {
public:
  Node* cloneGraph(Node* node) {
    unordered_map<Node*, Node*> g;
    auto dfs = [&](auto&& dfs, Node* node) ->
      Node* {
      if (!node) {
        return nullptr;
      }
      if (g.contains(node)) {
        return g[node];
      }
      Node* cloned = new Node(node->val);
      g[node] = cloned;
      for (auto& nxt : node->neighbors) {
        cloned->neighbors.push_back(dfs(dfs,
          nxt));
      }
      return cloned;
    };
    return dfs(dfs, node);
  }
};
```

03 LC 207 | Course Schedule (Medium)

Problem: There are a total of numCourses courses you have to take, labeled from 0 to numCourses - 1. You are given an array prerequisites where prerequisites[i] = [a_i, b_i] indicates that you must take course b_i first if you want to take course a_i. - For example, the pair [0, 1], indicates that to take course 0 you have to first take course 1. Return true if you can finish all courses. Otherwise, return false.

Constraints: 1 <= numCourses <= 2000; 0 <= prerequisites.length <= 5000; prerequisites[i].length == 2; 0 <= a_i, b_i < numCourses; All the pairs prerequisites[i] are unique.

```
class Solution {
public:
  bool canFinish(int numCourses,
    vector<vector<int>>& prerequisites) {
    vector<vector<int>> g(numCourses);
    vector<int> indeg(numCourses);
    for (auto& p : prerequisites) {
      int a = p[0], b = p[1];
      g[b].push_back(a);
      ++indeg[a];
    }
    queue<int> q;
    for (int i = 0; i < numCourses; ++i) {
      if (indeg[i] == 0) {
        q.push(i);
      }
    }
    while (!q.empty()) {
      int i = q.front();
      q.pop();
      --numCourses;
      for (int j : g[i]) {
        if (--indeg[j] == 0) {
          q.push(j);
        }
      }
    }
    return numCourses == 0;
  }
};
```

11 LC 785 | Is Graph Bipartite? (Medium)

Problem: Return whether all vertices of an undirected graph can be divided into two groups such that every edge joins opposite groups. The graph may be disconnected.

Constraints: graph.length == n; 1 <= n <= 100; 0 <= graph[u].length < n; 0 <= graph[u][i] <= n - 1; graph[u] does not contain u.; All the values of graph[u] are unique.; If graph[u] contains v, then graph[v] contains u.

```
class Solution {
public:
  bool isBipartite(vector<vector<int>>& graph) {
    int n = graph.size();
    vector<int> color(n);
    auto dfs = [&](auto&& dfs, int a, int c) ->
      bool {
      color[a] = c;
      for (int b : graph[a]) {
        if (color[b] == c || (color[b] == 0 &&
          !dfs(dfs, b, -c))) {
          return false;
        }
      }
      return true;
    };
    for (int i = 0; i < n; ++i) {
      if (color[i] == 0 && !dfs(dfs, i, 1)) {
        return false;
      }
    }
    return true;
  }
};
```

04 LC 210 | Course Schedule II (Medium)

Problem: Return an order for all numCourses courses such that every prerequisite appears before its dependent course. Pair [a,b] means b must precede a. Return an empty array if impossible; any valid order is accepted.

Constraints: 1 <= numCourses <= 2000; 0 <= prerequisites.length <= numCourses * (numCourses - 1); prerequisites[i].length == 2; 0 <= a_i, b_i < numCourses; a_i != b_i; All the pairs [a_i, b_i] are distinct.

```
class Solution {
public:
  vector<int> findOrder(int numCourses,
    vector<vector<int>>& prerequisites) {
    vector<vector<int>> g(numCourses);
    vector<int> indeg(numCourses);
    for (auto& p : prerequisites) {
      int a = p[0], b = p[1];
      g[b].push_back(a);
      ++indeg[a];
    }
    queue<int> q;
    for (int i = 0; i < numCourses; ++i) {
      if (indeg[i] == 0) {
        q.push(i);
      }
    }
    vector<int> ans;
    while (!q.empty()) {
      int i = q.front();
      q.pop();
      ans.push_back(i);
      for (int j : g[i]) {
        if (--indeg[j] == 0) {
          q.push(j);
        }
      }
    }
    return ans.size() == numCourses ? ans :
      vector<int>();
  }
};
```

13 LC 841 | Keys and Rooms (Medium)

Problem: Room 0 is open; all others are locked. Each visited room supplies keys to other rooms. Return whether every room can be visited from room 0.

Constraints: n == rooms.length; 2 <= n <= 1000; 0 <= rooms[i].length <= 1000; 1 <= sum(rooms[i].length) <= 3000; 0 <= rooms[i][j] < n; All the values of rooms[i] are unique.

```
class Solution {
public:
  bool canVisitAllRooms(vector<vector<int>>&
    rooms) {
    int n = rooms.size();
    vector<char> vis(n);
    vector<int> stk = {0};
    while (!stk.empty()) {
      int i = stk.back();
      stk.pop_back();
      if (vis[i]) {
        continue;
      }
      vis[i] = 1;
      for (int j : rooms[i]) {
        stk.push_back(j);
      }
    }
    for (char v : vis) {
      if (!v) {
        return false;
      }
    }
    return true;
  }
};
```

05 LC 994 | Rotting Oranges (Medium)

Problem: You are given an m x n grid where each cell can have one of three values: - 0 representing an empty cell, - 1 representing a fresh orange, or - 2 representing a rotten orange. Every minute, any fresh orange that is 4-directionally adjacent to a rotten orange becomes rotten. Return the minimum number of minutes that must elapse until no cell has a fresh orange. If this is impossible, return -1.

Constraints: m == grid.length; n == grid[i].length; 1 <= m, n <= 10; grid[i][j] is 0, 1, or 2.

```
class Solution {
public:
  int orangesRotting(vector<vector<int>>& grid) {
    int m = grid.size(), n = grid[0].size();
    queue<pair<int, int>> q;
    int cnt = 0;
    for (int i = 0; i < m; ++i) {
      for (int j = 0; j < n; ++j) {
        if (grid[i][j] == 1) {
          ++cnt;
        } else if (grid[i][j] == 2) {
          q.emplace(i, j);
        }
      }
    }
    const int dirs[5] = {-1, 0, 1, 0, -1};
    for (int ans = 1; q.size() && cnt; ++ans) {
      for (int k = q.size(); k; --k) {
        auto [i, j] = q.front();
        q.pop();
        for (int d = 0; d < 4; ++d) {
          int x = i + dirs[d], y = j + dirs[d +
            1];
          if (x >= 0 && x < m && y >= 0 && y < n
            && grid[x][y] == 1) {
            grid[x][y] = 2;
            q.emplace(x, y);
            if (--cnt == 0) {
              return ans;
            }
          }
        }
      }
    }
    return cnt > 0 ? -1 : 0;
  }
};
```

06 LC 547 | Number of Provinces (Medium)

Problem: isConnected is a symmetric adjacency matrix. A province is a connected component, including isolated cities. Return the number of provinces.

Constraints: 1 <= n <= 200; n == isConnected.length; n == isConnected[i].length; isConnected[i][j] is 1 or 0.; isConnected[i][i] == 1; isConnected[i][j] == isConnected[j][i]

```
class Solution {
public:
  int findCircleNum(vector<vector<int>>&
    isConnected) {
    int n = isConnected.size();
    int ans = 0;
    vector<char> vis = vector<char>(n, 0);
    auto dfs = [&](auto&& dfs, int i) -> void {
      vis[i] = true;
      for (int j = 0; j < n; ++j) {
        if (!vis[j] && isConnected[i][j]) {
          dfs(dfs, j);
        }
      }
    };
    for (int i = 0; i < n; ++i) {
      if (!vis[i]) {
        dfs(dfs, i);
        ++ans;
      }
    }
    return ans;
  }
};
```

39 LC 1162 | As Far from Land as Possible (Medium)

Problem: In a binary grid, 1 is land and 0 water. Return the maximum, over water cells, of the Manhattan distance to the nearest land. Return -1 if the grid is all land or all water.

Constraints: n == grid.length; n == grid[i].length; 1 <= n <= 100; grid[i][j] is 0 or 1

```
class Solution {
public:
  int maxDistance(vector<vector<int>>& grid) {
    int n = grid.size();
    queue<pair<int, int>> q;
    for (int i = 0; i < n; ++i) {
      for (int j = 0; j < n; ++j) {
        if (grid[i][j]) {
          q.emplace(i, j);
        }
      }
    }
    int ans = -1;
    if (q.empty() || q.size() == n * n) {
      return ans;
    }
    int dirs[5] = {-1, 0, 1, 0, -1};
    while (!q.empty()) {
      for (int m = q.size(); m; --m) {
        auto [i, j] = q.front();
        q.pop();
        for (int k = 0; k < 4; ++k) {
          int x = i + dirs[k], y = j + dirs[k +
            1];
          if (x >= 0 && x < n && y >= 0 && y < n
            && !grid[x][y]) {
            grid[x][y] = 1;
            q.emplace(x, y);
          }
        }
      }
      ++ans;
    }
    return ans;
  }
};
```

07 LC 695 | Max Area of Island (Medium)

Problem: You are given an m x n binary matrix grid. An island is a group of 1's (representing land) connected 4-directionally (horizontal or vertical.) You may assume all four edges of the grid are surrounded by water. The area of an island is the number of cells with a value 1 in the island. Return the maximum area of an island in grid. If there is no island, return 0.

Constraints: m == grid.length; n == grid[i].length; 1 <= m, n <= 50; grid[i][j] is either 0 or 1.

```
class Solution {
public:
  int maxAreaOfIsland(vector<vector<int>>& grid) {
    int m = grid.size(), n = grid[0].size();
    int dirs[5] = {-1, 0, 1, 0, -1};
    int ans = 0;
    function<int(int, int)> dfs = [&](int i, int
      j) {
      if (grid[i][j] == 0) {
        return 0;
      }
      int ans = 1;
      grid[i][j] = 0;
      for (int k = 0; k < 4; ++k) {
        int x = i + dirs[k], y = j + dirs[k + 1];
        if (x >= 0 && x < m && y >= 0 && y < n) {
          ans += dfs(x, y);
        }
      }
      return ans;
    };
    for (int i = 0; i < m; ++i) {
      for (int j = 0; j < n; ++j) {
        ans = max(ans, dfs(i, j));
      }
    }
    return ans;
  }
};
```

26 LC 733 | Flood Fill (Easy)

Problem: Starting at (sr,sc), recolor every cell connected by horizontal/vertical steps through the starting cell's original color. Return the modified image; cells of other colors and disconnected cells stay unchanged.

Constraints: m == image.length; n == image[i].length; 1 <= m, n <= 50; 0 <= image[i][j], color < 2^16; 0 <= sr < m; 0 <= sc < n

```
class Solution {
public:
  vector<vector<int>>
    floodFill(vector<vector<int>>& image, int sr,
    int sc, int color) {
    if (image[sr][sc] == color) return image;
    int oc = image[sr][sc];
    image[sr][sc] = color;
    queue<pair<int, int>> q;
    q.push({sr, sc});
    int dirs[5] = {-1, 0, 1, 0, -1};
    while (!q.empty()) {
      auto [a, b] = q.front();
      q.pop();
      for (int k = 0; k < 4; ++k) {
        int x = a + dirs[k];
        int y = b + dirs[k + 1];
        if (x >= 0 && x < image.size() && y >= 0
          && y < image[0].size() && image[x][y] ==
          oc) {
          q.push({x, y});
          image[x][y] = color;
        }
      }
    }
    return image;
  }
};
```

08 LC 417 | Pacific Atlantic Water Flow (Medium)

Problem: A height grid borders the Pacific at top/left and Atlantic at bottom/right. Water moves horizontally/vertically to equal or lower cells. Return all cells from which water can reach both oceans.
Constraints: m == heights.length; n == heights[r].length; 1 <= m, n <= 200; 0 <= heights[r][c] <= 10^5

```
class Solution {
public:
  vector<vector<int>>
    pacificAtlantic(vector<vector<int>>& heights)
    {
    int m = heights.size(), n = heights[0].size();
    vector<vector<bool>> vis1(m, vector<bool>(n,
      false)), vis2(m, vector<bool>(n, false));
    queue<pair<int, int>> q1, q2;
    vector<int> dirs = {-1, 0, 1, 0, -1};
    for (int i = 0; i < m; ++i) {
      q1.emplace(i, 0);
      vis1[i][0] = true;
      q2.emplace(i, n - 1);
      vis2[i][n - 1] = true;
    }
    for (int j = 0; j < n; ++j) {
      q1.emplace(0, j);
      vis1[0][j] = true;
      q2.emplace(m - 1, j);
      vis2[m - 1][j] = true;
    }
    auto bfs = [&](queue<pair<int, int>>& q,
      vector<vector<bool>>& vis) {
      while (!q.empty()) {
        auto [x, y] = q.front();
        q.pop();
        for (int k = 0; k < 4; ++k) {
          int nx = x + dirs[k], ny = y + dirs[k +
            1];
          if (nx >= 0 && nx < m && ny >= 0 && ny <
            n
            && !vis[nx][ny]
            && heights[nx][ny] >= heights[x][y]) {
            vis[nx][ny] = true;
            q.emplace(nx, ny);
          }
        }
      }
    };
    bfs(q1, vis1);
    bfs(q2, vis2);
    vector<vector<int>> ans;
    for (int i = 0; i < m; ++i)
      for (int j = 0; j < n; ++j)
        if (vis1[i][j] && vis2[i][j])
          ans.push_back({i, j});
    return ans;
  }
};
```

09 LC 127 | Word Ladder (Hard)

Problem: Return the number of words in a shortest transformation from beginWord to endWord, changing one letter per step. Every subsequent word must be in wordList; beginWord need not. Return 0 if impossible.
Constraints: 1 <= beginWord.length <= 10; endWord.length == beginWord.length; 1 <= wordList.length <= 5000; wordList[i].length == beginWord.length; beginWord, endWord, and wordList[i] consist of lowercase English letters.; beginWord != endWord; All the words in wordList are unique.

```
class Solution {
public:
  int ladderLength(string beginWord, string
    endWord, vector<string>& wordList) {
    unordered_set<string> words(wordList.begin(),
      wordList.end());
    if (!words.count(endWord)) return 0;
    queue<string> q1{{beginWord}};
    queue<string> q2{{endWord}};
    unordered_map<string, int> m1;
    unordered_map<string, int> m2;
    m1[beginWord] = 0;
    m2[endWord] = 0;
    while (!q1.empty() && !q2.empty()) {
      int t = q1.size() <= q2.size() ? extend(m1,
        m2, q1, words) : extend(m2, m1, q2,
        words);
      if (t != -1) return t + 1;
    }
    return 0;
  }
  int extend(unordered_map<string, int>& m1,
    unordered_map<string, int>& m2, queue<string>&
    q, unordered_set<string>& words) {
    for (int i = q.size(); i > 0; --i) {
      string s = q.front();
      int step = m1[s];
      q.pop();
      for (int j = 0; j < s.size(); ++j) {
        char ch = s[j];
        for (char k = 'a'; k <= 'z'; ++k) {
          s[j] = k;
          if (!words.count(s) || m1.count(s))
            continue;
          if (m2.count(s)) return step + 1 +
            m2[s];
          m1[s] = step + 1;
          q.push(s);
        }
        s[j] = ch;
      }
    }
    return -1;
  }
};
```

10 LC 752 | Open the Lock (Medium)

Problem: A four-wheel lock starts at '0000'. Each move turns one wheel one step forward or backward with digit wraparound. Avoid every deadend, including the starting state. Return minimum moves to target, or -1 if impossible.

Constraints: 1 <= deadends.length <= 500; deadends[i].length == 4; target.length == 4; target will not be in the list deadends.; target and deadends[i] consist of digits only.

```
class Solution {
public:
  int openLock(vector<string>& deadends, string
    target) {
    unordered_set<string> s(deadends.begin(),
      deadends.end());
    if (s.count("0000")) return -1;
    if (target == "0000") return 0;
    queue<string> q{{"0000"}};
    s.insert("0000");
    int ans = 0;
    while (!q.empty()) {
      ++ans;
      for (int n = q.size(); n > 0; --n) {
        string p = q.front();
        q.pop();
        for (string t : next(p)) {
          if (target == t) return ans;
          if (!s.count(t)) {
            q.push(t);
            s.insert(t);
          }
        }
      }
    }
    return -1;
  }
  vector<string> next(string& t) {
    vector<string> res;
    for (int i = 0; i < 4; ++i) {
      char c = t[i];
      t[i] = c == '0' ? '9' : (char) (c - 1);
      res.push_back(t);
      t[i] = c == '9' ? '0' : (char) (c + 1);
      res.push_back(t);
      t[i] = c;
    }
    return res;
  }
};
```

12 LC 797 | All Paths From Source to Target (Medium)

Problem: Given a directed acyclic graph (DAG) of n nodes labeled from 0 to n - 1, find all possible paths from node 0 to node n - 1 and return them in any order. The graph is given as follows: graph[i] is a list of all nodes you can visit from node i (i.e., there is a directed edge from node i to node graph[i][j]).

Constraints: n == graph.length; 2 <= n <= 15; 0 <= graph[i][j] < n; graph[i][j] != i (i.e., there will be no self-loops).; All the elements of graph[i] are unique.; The input graph is guaranteed to be a DAG.

```
class Solution {
public:
  vector<vector<int>>
    allPathsSourceTarget(vector<vector<int>>&
    graph) {
    vector<vector<int>> result;
    vector<int> path{0};
    auto dfs = [&](auto&& self, int u) -> void {
      if (u == static_cast<int>(graph.size()) - 1)
        { result.push_back(path); return; }
      for (int v : graph[u]) {
        path.push_back(v); self(self, v);
          path.pop_back();
      }
    };
    dfs(dfs, 0);
    return result;
  }
};
```

45 LC 1376 | Time Needed to Inform All Employees (Medium)

Problem: Employees form a management tree rooted at headID. An informed employee needs informTime[i] minutes to inform all direct reports, after which they can spread the news concurrently. Return when everyone is informed.

Constraints: 1 <= n <= 10^5; 0 <= headID < n; manager.length == n; 0 <= manager[i] < n; manager[headID] == -1; informTime.length == n; 0 <= informTime[i] <= 1000; informTime[i] == 0 if employee i has no subordinates.; It is guaranteed that all the employees can be informed.

```
class Solution {
public:
  int numOfMinutes(int n, int headID, vector<int>&
    manager, vector<int>& informTime) {
    vector<vector<int>> g(n);
    for (int i = 0; i < n; ++i) {
      if (manager[i] >= 0) {
        g[manager[i]].push_back(i);
      }
    }
    vector<int> time(n);
    vector<array<int, 2>> stk{{headID, 0}};
    while (!stk.empty()) {
      auto [i, state] = stk.back();
      stk.pop_back();
      if (state == 0) {
        stk.push_back({i, 1});
        for (int j : g[i]) {
          stk.push_back({j, 0});
        }
      } else {
        int ans = 0;
        for (int j : g[i]) {
          ans = max(ans, time[j] + informTime[i]);
        }
        time[i] = ans;
      }
    }
    return time[headID];
  }
};
```

14 LC 743 | Network Delay Time (Medium)

Problem: In a directed weighted graph, a signal starts at k. Return the earliest time by which every node has received it, or -1 if any node is unreachable.
Constraints: 1 <= k <= n <= 100; 1 <= times.length <= 6000; times[i].length == 3; 1 <= u_i, v_i <= n; u_i != v_i; 0 <= w_i <= 100; All the pairs (u_i, v_i) are unique. (i.e., no multiple edges.)

```
class Solution {
public:
  int networkDelayTime(vector<vector<int>>& times,
    int n, int k) {
    const int inf = 1 << 29;
    using pii = pair<int, int>;
    vector<vector<pii>> g(n);
    for (auto& edge : times) {
      g[edge[0] - 1].emplace_back(edge[1] - 1,
        edge[2]);
    }
    vector<int> dist(n, inf);
    dist[k - 1] = 0;
    priority_queue<pii, vector<pii>, greater<>>
      pq;
    pq.emplace(0, k - 1);
    while (!pq.empty()) {
      auto [d, u] = pq.top();
      pq.pop();
      if (d > dist[u]) {
        continue;
      }
      for (auto& [v, w] : g[u]) {
        if (dist[v] > dist[u] + w) {
          dist[v] = dist[u] + w;
          pq.emplace(dist[v], v);
        }
      }
    }
    int ans = ranges::max(dist);
    return ans == inf ? -1 : ans;
  }
};
```

29 LC 1466 l Reorder Routes to Make All Paths Lead to the Citv Zero (Medium)

Problem: The n-vertex road network is a tree with directed edges. Reverse the fewest roads so every city can reach city 0 following edge directions. Return the number of reversals.
Constraints: 2 <= n <= 5 * 10^4; connections.length == n - 1; connections[i].length == 2; 0 <= a_i, b_i <= n - 1; a_i != b_i

```
class Solution {
public:
  int minReorder(int n, vector<vector<int>>&
    connections) {
    vector<vector<pair<int, int>>> g(n);
    for (auto& e : connections) {
      int a = e[0], b = e[1];
      g[a].emplace_back(b, 1);
      g[b].emplace_back(a, 0);
    }
    int ans = 0;
    vector<pair<int, int>> stk{{0, -1}};
    while (!stk.empty()) {
      auto cur = stk.back();
      stk.pop_back();
      int a = cur.first, fa = cur.second;
      for (auto& [b, c] : g[a]) {
        if (b != fa) {
          ans += c;
          stk.emplace_back(b, a);
        }
      }
    }
    return ans;
  }
};
```

15 LC 787 | Cheapest Flights Within K Stops (Medium)

Problem: There are n cities connected by some number of flights. You are given an array flights where flights[i] = [from_i, to_i, price_i] indicates that there is a flight from city from_i to city to_i with cost price_i. You are also given three integers src, dst, and k, return the cheapest price from src to dst with at most k stops. If there is no such route, return -1.
Constraints: 2 <= n <= 100; 0 <= flights.length <= (n * (n - 1) / 2); flights[i].length == 3; 0 <= from_i, to_i < n; from_i != to_i; 1 <= price_i <= 10^4; There will not be any multiple flights between two cities.; 0 <= src, dst, k < n; src != dst

```
class Solution {
public:
  int findCheapestPrice(int n,
    vector<vector<int>>& flights, int src, int
    dst, int k) {
    const int inf = 0x3f3f3f3f;
    vector<int> dist(n, inf);
    vector<int> backup;
    dist[src] = 0;
    for (int i = 0; i < k + 1; ++i) {
      backup = dist;
      for (auto& e : flights) {
        int f = e[0], t = e[1], p = e[2];
        dist[t] = min(dist[t], backup[f] + p);
      }
    }
    return dist[dst] == inf ? -1 : dist[dst];
  }
};
```

31 LC 886 | Possible Bipartition (Medium)

Problem: Split people labeled 1..n into two groups so every listed dislike pair lies in different groups. Return whether a valid split exists.
Constraints: 1 <= n <= 2000; 0 <= dislikes.length <= 10^4; dislikes[i].length == 2; 1 <= a_i < b_i <= n; All the pairs of dislikes are unique.

```
class Solution {
public:
  bool possibleBipartition(int n,
    vector<vector<int>>& dislikes) {
    vector<vector<int>> g(n);
    for (auto& e : dislikes) {
      int a = e[0] - 1, b = e[1] - 1;
      g[a].push_back(b);
      g[b].push_back(a);
    }
    vector<int> color(n);
    for (int start = 0; start < n; ++start) {
      if (color[start]) {
        continue;
      }
      color[start] = 1;
      vector<int> stk{start};
      while (!stk.empty()) {
        int i = stk.back();
        stk.pop_back();
        for (int j : g[i]) {
          if (color[j] == color[i]) {
            return false;
          }
          if (color[j] == 0) {
            color[j] = 3 - color[i];
            stk.push_back(j);
          }
        }
      }
    }
    return true;
  }
};
```

16 LC 1631 | Path With Minimum Effort (Medium)

Problem: Travel from top-left to bottom-right of heights using horizontal/vertical moves. A route's effort is its largest absolute height difference across one step. Return the minimum achievable effort.

Constraints: rows == heights.length; columns == heights[i].length; 1 <= rows, columns <= 100; 1 <= heights[i][j] <= 10^6

```
class Solution {
public:
  int minimumEffortPath(vector<vector<int>>&
    heights) {
    int m = heights.size(), n = heights[0].size();
    vector<vector<int>> dist =
      vector<vector<int>>(m, vector<int>(n,
      0x3f3f3f3f));
    dist[0][0] = 0;
    int dirs[5] = {0, 1, 0, -1, 0};
    using T = tuple<int, int, int>;
    priority_queue<T, vector<T>, greater<T>> pq;
    pq.emplace(0, 0, 0);
    while (!pq.empty()) {
      auto [t, i, j] = pq.top();
      pq.pop();
      for (int k = 0; k < 4; ++k) {
        int x = i + dirs[k], y = j + dirs[k + 1];
        if (x < 0 || x >= m || y < 0 || y >= n) {
          continue;
        }
        int d = max(t, abs(heights[x][y] -
          heights[i][j]));
        if (d < dist[x][y]) {
          dist[x][y] = d;
          pq.emplace(d, x, y);
        }
      }
    }
    return dist[m - 1][n - 1];
  }
};
```

30 LC 802 | Find Eventual Safe States (Medium)

Problem: A directed-graph node is safe if every path starting there eventually reaches a terminal node rather than a cycle. Return all safe nodes in ascending order.

Constraints: n == graph.length; 1 <= n <= 10^4; 0 <= graph[i].length <= n; 0 <= graph[i][j] <= n - 1; graph[i] is sorted in a strictly increasing order.; The graph may contain self-loops.; The number of edges in the graph will be in the range [1, 4 * 10^4].

```
class Solution {
public:
  vector<int>
    eventualSafeNodes(vector<vector<int>>& graph)
    {
    int n = graph.size();
    vector<int> indeg(n);
    vector<vector<int>> rg(n);
    queue<int> q;
    for (int i = 0; i < n; ++i) {
      for (int j : graph[i]) rg[j].push_back(i);
      indeg[i] = graph[i].size();
      if (indeg[i] == 0) q.push(i);
    }
    while (!q.empty()) {
      int i = q.front();
      q.pop();
      for (int j : rg[i])
        if (--indeg[j] == 0) q.push(j);
    }
    vector<int> ans;
    for (int i = 0; i < n; ++i)
      if (indeg[i] == 0) ans.push_back(i);
    return ans;
  }
};
```

17 LC 778 | Swim in Rising Water (Hard)

Problem: At time t, grid cells of elevation at most t are passable. Move horizontally/vertically between passable cells with no travel-time cost. Return the earliest t permitting travel from top-left to bottom-right.

Constraints: n == grid.length; n == grid[i].length; 1 <= n <= 50; 0 <= grid[i][j] < n^2; Each value grid[i][j] is unique.

```
class Solution {
public:
  int swimInWater(vector<vector<int>>& grid) {
    int n = grid.size();
    int m = n * n;
    vector<int> p(m);
    iota(p.begin(), p.end(), 0);
    auto find = [&](auto&& find, int x) -> int {
      if (p[x] != x) {
        p[x] = find(find, p[x]);
      }
      return p[x];
    };
    vector<int> hi(m);
    for (int i = 0; i < n; ++i) {
      for (int j = 0; j < n; ++j) {
        hi[grid[i][j]] = i * n + j;
      }
    }
    array<int, 5> dirs{-1, 0, 1, 0, -1};
    for (int t = 0; t < m; ++t) {
      int id = hi[t];
      int x = id / n, y = id % n;
      for (int k = 0; k < 4; ++k) {
        int nx = x + dirs[k], ny = y + dirs[k +
          1];
        if (nx >= 0 && nx < n && ny >= 0 && ny < n
          && grid[nx][ny] <= t) {
          int a = find(find, x * n + y);
          int b = find(find, nx * n + ny);
          p[a] = b;
        }
      }
      if (find(find, 0) == find(find, m - 1)) {
        return t;
      }
    }
    return 0;
  }
};
```

18 LC 1584 | Min Cost to Connect All Points (Medium)

Problem: Connect all points with edges costing their Manhattan distance. Return the minimum total cost of a connected network; no cycles are needed.

Constraints: 1 <= points.length <= 1000; -10^6 <= x_i, y_i <= 10^6; All pairs (x_i, y_i) are distinct.

```
class Solution {
public:
  int minCostConnectPoints(vector<vector<int>>&
    points) {
    int n = points.size();
    vector<int> distance(n, INT_MAX);
    vector<bool> used(n);
    distance[0] = 0;
    int cost = 0;
    for (int count = 0; count < n; ++count) {
      int v = -1;
      for (int i = 0; i < n; ++i)
        if (!used[i] && (v == -1 || distance[i] <
          distance[v])) v = i;
      used[v] = true;
      cost += distance[v];
      for (int i = 0; i < n; ++i)
        if (!used[i]) distance[i] =
          min(distance[i],
          abs(points[i][0] - points[v][0]) +
            abs(points[i][1] - points[v][1]));
    }
    return cost;
  }
};
```

25 LC 130 | Surrounded Regions (Medium)

Problem: In a board of 'X' and 'O', replace every 'O' region not connected by horizontal/vertical 'O' steps to the boundary with 'X'. Modify the board in place.

Constraints: m == board.length; n == board[i].length; 1 <= m, n <= 200; board[i][j] is 'X' or 'O'.

```
class Solution {
public:
  void solve(vector<vector<char>>& board) {
    int m = board.size(), n = board[0].size();
    int dirs[5] = {-1, 0, 1, 0, -1};
    function<void(int, int)> dfs = [&](int i, int
      j) {
      if (i < 0 || i >= m || j < 0 || j >= n ||
        board[i][j] != 'O') {
        return;
      }
      board[i][j] = '.';
      for (int k = 0; k < 4; ++k) {
        dfs(i + dirs[k], j + dirs[k + 1]);
      }
    };
    for (int i = 0; i < m; ++i) {
      dfs(i, 0);
      dfs(i, n - 1);
    }
    for (int j = 1; j < n - 1; ++j) {
      dfs(0, j);
      dfs(m - 1, j);
    }
    for (auto& row : board) {
      for (auto& c : row) {
        if (c == '.') {
          c = 'O';
        } else if (c == 'O') {
          c = 'X';
        }
      }
    }
  }
};
```

19 LC 684 | Redundant Connection (Medium)

Problem: An undirected graph was a tree before one extra edge was added. Return an edge whose removal restores a tree. If several qualify, choose the one appearing last in the input.

Constraints: n == edges.length; 3 <= n <= 1000; edges[i].length == 2; 1 <= a_i < b_i <= edges.length; a_i != b_i; There are no repeated edges.; The given graph is connected.

```
class Solution {
public:
  vector<int>
    findRedundantConnection(vector<vector<int>>&
    edges) {
    int n = edges.size();
    vector<int> p(n);
    iota(p.begin(), p.end(), 0);
    function<int(int)> find = [&](int x) {
      return x == p[x] ? x : p[x] = find(p[x]);
    };
    for (int i = 0;; ++i) {
      int pa = find(edges[i][0] - 1);
      int pb = find(edges[i][1] - 1);
      if (pa == pb) {
        return edges[i];
      }
      p[pa] = pb;
    }
  }
};
```

48 LC 2368 | Reachable Nodes With Restrictions (Medium)

Problem: In an undirected tree rooted at 0, restricted vertices cannot be visited. Return how many vertices, including 0, are reachable without entering a restricted vertex; 0 is not restricted.

Constraints: 2 <= n <= 10^5; edges.length == n - 1; edges[i].length == 2; 0 <= a_i, b_i < n; a_i != b_i; edges represents a valid tree.; 1 <= restricted.length < n; 1 <= restricted[i] < n; All the values of restricted are unique.

```
class Solution {
public:
  int reachableNodes(int n, vector<vector<int>>&
    edges, vector<int>& restricted) {
    vector<vector<int>> g(n);
    vector<int> vis(n);
    for (auto& e : edges) {
      int a = e[0], b = e[1];
      g[a].emplace_back(b);
      g[b].emplace_back(a);
    }
    for (int i : restricted) {
      vis[i] = true;
    }
    int ans = 0;
    vector<int> stk{0};
    while (!stk.empty()) {
      int i = stk.back();
      stk.pop_back();
      if (vis[i]) {
        continue;
      }
      vis[i] = true;
      ++ans;
      for (int j : g[i]) {
        if (!vis[j]) {
          stk.emplace_back(j);
        }
      }
    }
    return ans;
  }
};
```

20 LC 721 | Accounts Merge (Medium)

Problem: Each account contains a name followed by emails. Merge accounts connected through shared emails; equal names alone do not identify a person. All accounts of one person share a name. Return each merged name followed by sorted emails, with merged accounts in any order.

Constraints: 1 <= accounts.length <= 1000; 2 <= accounts[i].length <= 10; 1 <= accounts[i][j].length <= 30; accounts[i][0] consists of English letters.; accounts[i][j] (for j > 0) is a valid email.

```
class UnionFind {
public:
  UnionFind(int n) {
    p = vector<int>(n);
    size = vector<int>(n, 1);
    iota(p.begin(), p.end(), 0);
  }
  bool unite(int a, int b) {
    int pa = find(a), pb = find(b);
    if (pa == pb) {
      return false;
    }
    if (size[pa] > size[pb]) {
      p[pb] = pa;
      size[pa] += size[pb];
    } else {
      p[pa] = pb;
      size[pb] += size[pa];
    }
    return true;
  }
  int find(int x) {
    if (p[x] != x) {
      p[x] = find(p[x]);
    }
    return p[x];
  }
private:
  vector<int> p, size;
};
class Solution {
public:
  vector<vector<string>>
    accountsMerge(vector<vector<string>>&
    accounts) {
    int n = accounts.size();
    UnionFind uf(n);
    unordered_map<string, int> d;
    for (int i = 0; i < n; ++i) {
      for (int j = 1; j < accounts[i].size(); ++j)
        {
        const string& email = accounts[i][j];
        if (d.find(email) != d.end()) {
          uf.unite(i, d[email]);
        } else {
          d[email] = i;
        }
      }
    }
    unordered_map<int, set<string>> g;
    for (int i = 0; i < n; ++i) {
      int root = uf.find(i);
      g[root].insert(accounts[i].begin() + 1,
        accounts[i].end());
    }
    vector<vector<string>> ans;
    for (const auto& [root, s] : g) {
      vector<string> emails(s.begin(), s.end());
      emails.insert(emails.begin(),
        accounts[root][0]);
      ans.push_back(emails);
    }
    return ans;
  }
};
```

21 LC 399 | Evaluate Division (Medium)

Problem: Equations give positive ratios equations[i][0]/equations[i][1]=values[i]. Return each queried ratio, or -1.0 when it cannot be determined. Undefined variables, including an undefined variable divided by itself, yield -1.0; inputs are consistent.

Constraints: 1 <= equations.length <= 20; equations[i].length == 2; 1 <= A_i.length, B_i.length <= 5; values.length == equations.length; 0.0 < values[i] <= 20.0; 1 <= queries.length <= 20; queries[i].length == 2; 1 <= C_j.length, D_j.length <= 5; A_i, B_i, C_j, D_j consist of lower case English letters and digits.

```
class Solution {
public:
  unordered_map<string, string> p;
  unordered_map<string, double> w;
  vector<double>
    calcEquation(vector<vector<string>>&
    equations, vector<double>& values,
    vector<vector<string>>& queries) {
    int n = equations.size();
    for (auto e : equations) {
      p[e[0]] = e[0];
      p[e[1]] = e[1];
      w[e[0]] = 1.0;
      w[e[1]] = 1.0;
    }
    for (int i = 0; i < n; ++i) {
      vector<string> e = equations[i];
      string a = e[0], b = e[1];
      string pa = find(a), pb = find(b);
      if (pa == pb) continue;
      p[pa] = pb;
      w[pa] = w[b] * values[i] / w[a];
    }
    int m = queries.size();
    vector<double> ans(m);
    for (int i = 0; i < m; ++i) {
      string c = queries[i][0], d = queries[i][1];
      ans[i] = p.find(c) == p.end() || p.find(d)
        == p.end() || find(c) != find(d) ? -1.0 :
        w[c] / w[d];
    }
    return ans;
  }
  string find(string x) {
    if (p[x] != x) {
      string origin = p[x];
      p[x] = find(p[x]);
      w[x] *= w[origin];
    }
    return p[x];
  }
};
```

22 LC 909 | Snakes and Ladders (Medium)

Problem: Squares 1..n^2 run in alternating directions across rows, starting at bottom-left. From square 1, each roll advances 1..6 without overshooting; if the landing square has a snake/ladder, take exactly that one jump. Return minimum rolls to n^2, or -1.
Constraints: n == board.length == board[i].length; 2 <= n <= 20; board[i][j] is either -1 or in the range [1, n^2].; The squares labeled 1 and n^2 are not the starting points of any snake or ladder.

```
class Solution {
public:
  int snakesAndLadders(vector<vector<int>>& board)
    {
    int n = board.size();
    queue<int> q{{1}};
    int m = n * n;
    vector<bool> vis(m + 1);
    vis[1] = true;
    for (int ans = 0; !q.empty(); ++ans) {
      for (int k = q.size(); k > 0; --k) {
        int x = q.front();
        q.pop();
        if (x == m) {
          return ans;
        }
        for (int y = x + 1; y <= min(x + 6, m);
          ++y) {
          int i = (y - 1) / n, j = (y - 1) % n;
          if (i % 2 == 1) {
            j = n - j - 1;
          }
          i = n - i - 1;
          int z = board[i][j] == -1 ? y :
            board[i][j];
          if (!vis[z]) {
            vis[z] = true;
            q.push(z);
          }
        }
      }
    }
    return -1;
  }
};
```

23 LC 1091 | Shortest Path in Binary Matrix (Medium)

Problem: Return the length, counting cells, of a shortest path of zero cells from top-left to bottom-right in a binary square grid. Eight-direction moves are allowed. Return -1 if no clear path exists.
Constraints: n == grid.length; n == grid[i].length; 1 <= n <= 100; grid[i][j] is 0 or 1

```
class Solution {
public:
  int
    shortestPathBinaryMatrix(vector<vector<int>>&
    grid) {
    if (grid[0][0]) {
      return -1;
    }
    int n = grid.size();
    grid[0][0] = 1;
    queue<pair<int, int>> q;
    q.emplace(0, 0);
    for (int ans = 1; !q.empty(); ++ans) {
      for (int k = q.size(); k; --k) {
        auto [i, j] = q.front();
        q.pop();
        if (i == n - 1 && j == n - 1) {
          return ans;
        }
        for (int x = i - 1; x <= i + 1; ++x) {
          for (int y = j - 1; y <= j + 1; ++y) {
            if (x >= 0 && x < n && y >= 0 && y < n
              && !grid[x][y]) {
              grid[x][y] = 1;
              q.emplace(x, y);
            }
          }
        }
      }
    }
    return -1;
  }
};
```

32 LC 990 | Satisfiability of Equality Equations (Medium)

Problem: You are given an array of strings equations that represent relationships between variables where each string equations[i] is of length 4 and takes one of two different forms: "x_i==y_i" or "x_i!=y_i".Here, x_i and y_i are lowercase letters (not necessarily different) that represent one-letter variable names. Return true if it is possible to assign integers to variable names so as to satisfy all the given equations, or false otherwise.
Constraints: 1 <= equations.length <= 500; equations[i].length == 4; equations[i][0] is a lowercase letter.; equations[i][1] is either '=' or '!'.; equations[i][2] is '='.; equations[i][3] is a lowercase letter.

```
class Solution {
public:
  vector<int> p;
  bool equationsPossible(vector<string>&
    equations) {
    p.resize(26);
    for (int i = 0; i < 26; ++i) p[i] = i;
    for (auto& e : equations) {
      int a = e[0] - 'a', b = e[3] - 'a';
      if (e[1] == '=') p[find(a)] = find(b);
    }
    for (auto& e : equations) {
      int a = e[0] - 'a', b = e[3] - 'a';
      if (e[1] == '!' && find(a) == find(b))
        return false;
    }
    return true;
  }
  int find(int x) {
    if (p[x] != x) p[x] = find(p[x]);
    return p[x];
  }
};
```

24 LC 542 | 01 Matrix (Medium)

Problem: Given an m x n binary matrix mat, return the distance of the nearest 0 for each cell. The distance between two cells sharing a common edge is 1.

Constraints: m == mat.length; n == mat[i].length; 1 <= m, n <= 10^4; 1 <= m * n <= 10^4; mat[i][j] is either 0 or 1.; There is at least one 0 in mat.

```
class Solution {
public:
  vector<vector<int>>
    updateMatrix(vector<vector<int>>& mat) {
    int m = mat.size(), n = mat[0].size();
    vector<vector<int>> ans(m, vector<int>(n,
      -1));
    queue<pair<int, int>> q;
    for (int i = 0; i < m; ++i) {
      for (int j = 0; j < n; ++j) {
        if (mat[i][j] == 0) {
          ans[i][j] = 0;
          q.emplace(i, j);
        }
      }
    }
    vector<int> dirs = {-1, 0, 1, 0, -1};
    while (!q.empty()) {
      auto p = q.front();
      q.pop();
      for (int i = 0; i < 4; ++i) {
        int x = p.first + dirs[i];
        int y = p.second + dirs[i + 1];
        if (x >= 0 && x < m && y >= 0 && y < n &&
          ans[x][y] == -1) {
          ans[x][y] = ans[p.first][p.second] + 1;
          q.emplace(x, y);
        }
      }
    }
    return ans;
  }
};
```

27 LC 1971 | Find if Path Exists in Graph (Easy)

Problem: Given an undirected graph on vertices 0..n-1, return whether any path connects source to destination. Edges can be traversed in either direction.

Constraints: 1 <= n <= 2 * 10^5; 0 <= edges.length <= 2 * 10^5; edges[i].length == 2; 0 <= u_i, v_i <= n - 1; u_i != v_i; 0 <= source, destination <= n - 1; There are no duplicate edges.; There are no self edges.

```
class Solution {
public:
  bool validPath(int n, vector<vector<int>>&
    edges, int source, int destination) {
    if (source == destination) {
      return true;
    }
    vector<vector<int>> g(n);
    for (const auto& e : edges) {
      int u = e[0], v = e[1];
      g[u].push_back(v);
      g[v].push_back(u);
    }
    vector<char> vis(n);
    vis[source] = 1;
    vector<int> stk{source};
    while (!stk.empty()) {
      int i = stk.back();
      stk.pop_back();
      for (int j : g[i]) {
        if (j == destination) {
          return true;
        }
        if (!vis[j]) {
          vis[j] = 1;
          stk.push_back(j);
        }
      }
    }
    return false;
  }
};
```

34 LC 332 | Reconstruct Itinerary (Hard)

Problem: Use every directed flight ticket exactly once to construct an itinerary starting at 'JFK'. If several itineraries exist, return the lexicographically smallest. At least one valid itinerary exists.

Constraints: 1 <= tickets.length <= 300; tickets[i].length == 2; from_i.length == 3; to_i.length == 3; from_i and to_i consist of uppercase English letters.; from_i != to_i

```
class Solution {
public:
  vector<string>
    findItinerary(vector<vector<string>>& tickets)
    {
    sort(tickets.rbegin(), tickets.rend());
    unordered_map<string, vector<string>> g;
    for (const auto& ticket : tickets) {
      g[ticket[0]].push_back(ticket[1]);
    }
    vector<string> ans;
    auto dfs = [&](auto&& dfs, string& f) -> void
      {
      while (!g[f].empty()) {
        string t = g[f].back();
        g[f].pop_back();
        dfs(dfs, t);
      }
      ans.emplace_back(f);
    };
    string f = "JFK";
    dfs(dfs, f);
    ranges::reverse(ans);
    return ans;
  }
};
```

49 LC 1557 l Minimum Number of Vertices to Reach All Nodes (Medium)

Problem: Given a directed acyclic graph, with n vertices numbered from 0 to n-1, and an array edges where edges[i] = [from_i, to_i] represents a directed edge from node from_i to node to_i. Find the smallest set of vertices from which all nodes in the graph are reachable. It's guaranteed that a unique solution exists. Notice that you can return the vertices in any order.

Constraints: 2 <= n <= 10^5; 1 <= edges.length <= min(10^5, n * (n - 1) / 2); edges[i].length == 2; 0 <= from_i, to_i < n; All pairs (from_i, to_i) are distinct.

```
class Solution {
public:
  vector<int> findSmallestSetOfVertices(int n,
    vector<vector<int>>& edges) {
    vector<int> cnt(n);
    for (auto& e : edges) {
      ++cnt[e[1]];
    }
    vector<int> ans;
    for (int i = 0; i < n; ++i) {
      if (cnt[i] == 0) {
        ans.push_back(i);
      }
    }
    return ans;
  }
};
```

28 LC 1319 | Number of Operations to Make Network Connected (Medium)

Problem: There are n computers and existing undirected cables. An operation relocates one existing cable to connect a different pair. Return the minimum operations needed to connect all computers, or -1 if impossible.
Constraints: 1 <= n <= 10^5; 1 <= connections.length <= min(n * (n - 1) / 2, 10^5); connections[i].length == 2; 0 <= a_i, b_i < n; a_i != b_i; There are no repeated connections.; No two computers are connected by more than one cable.

```
class Solution {
public:
  int makeConnected(int n, vector<vector<int>>&
    connections) {
    vector<int> p(n);
    iota(p.begin(), p.end(), 0);
    int cnt = 0;
    function<int(int)> find = [&](int x) -> int {
      if (p[x] != x) {
        p[x] = find(p[x]);
      }
      return p[x];
    };
    for (const auto& c : connections) {
      int pa = find(c[0]), pb = find(c[1]);
      if (pa == pb) {
        ++cnt;
      } else {
        p[pa] = pb;
        --n;
      }
    }
    return cnt >= n - 1 ? n - 1 : -1;
  }
};
```

50 LC 1765 | Map of Highest Peak (Medium)

Problem: Assign nonnegative grid heights: water cells have height 0, and horizontal/vertical neighbors differ by at most 1. Maximize the largest height and return any optimal height grid.
Constraints: m == isWater.length; n == isWater[i].length; 1 <= m, n <= 1000; isWater[i][j] is 0 or 1.; There is at least one water cell.

```
class Solution {
public:
  const int dirs[5] = {-1, 0, 1, 0, -1};
  vector<vector<int>>
    highestPeak(vector<vector<int>>& isWater) {
    int m = isWater.size(), n = isWater[0].size();
    vector<vector<int>> ans(m, vector<int>(n));
    queue<pair<int, int>> q;
    for (int i = 0; i < m; ++i) {
      for (int j = 0; j < n; ++j) {
        ans[i][j] = isWater[i][j] - 1;
        if (ans[i][j] == 0) {
          q.emplace(i, j);
        }
      }
    }
    while (!q.empty()) {
      auto [i, j] = q.front();
      q.pop();
      for (int k = 0; k < 4; ++k) {
        int x = i + dirs[k], y = j + dirs[k + 1];
        if (x >= 0 && x < m && y >= 0 && y < n &&
          ans[x][y] == -1) {
          ans[x][y] = ans[i][j] + 1;
          q.emplace(x, y);
        }
      }
    }
    return ans;
  }
};
```

33 LC 1192 | Critical Connections in a Network (Hard)

Problem: There are n servers numbered from 0 to n - 1 connected by undirected server-to-server connections forming a network where connections[i] = [a_i, b_i] represents a connection between servers a_i and b_i. Any server can reach other servers directly or indirectly through the network. A critical connection is a connection that, if removed, will make some servers unable to reach some other server. Return all critical connections in the network in any order.
Constraints: 2 <= n <= 10^5; n - 1 <= connections.length <= 10^5; 0 <= a_i, b_i <= n - 1; a_i != b_i; There are no repeated connections.

```
class Solution {
public:
  vector<vector<int>> criticalConnections(int n,
    vector<vector<int>>& connections) {
    vector<vector<pair<int, int>>> graph(n);
    for (int id = 0; id <
      static_cast<int>(connections.size()); ++id)
      {
      int a = connections[id][0], b =
        connections[id][1];
      graph[a].emplace_back(b, id);
        graph[b].emplace_back(a, id);
    }
    vector<int> discovery(n, -1), low(n),
      parent(n, -1), parentEdge(n, -1), next(n);
    vector<vector<int>> result;
    int timer = 0;
    for (int start = 0; start < n; ++start) {
      if (discovery[start] != -1) continue;
      vector<int> stack{start};
      discovery[start] = low[start] = timer++;
      while (!stack.empty()) {
        int u = stack.back();
        if (next[u] <
          static_cast<int>(graph[u].size())) {
          auto [v, edge] = graph[u][next[u]++];
          if (edge == parentEdge[u]) continue;
          if (discovery[v] == -1) {
            parent[v] = u; parentEdge[v] = edge;
            discovery[v] = low[v] = timer++;
            stack.push_back(v);
          } else low[u] = min(low[u],
            discovery[v]);
        } else {
          stack.pop_back();
          if (parent[u] != -1) {
            int p = parent[u];
            if (low[u] > discovery[p])
              result.push_back({p, u});
            low[p] = min(low[p], low[u]);
          }
        }
      }
    }
    return result;
  }
};
```

35 LC 269 | Alien Dictionary (Hard)

Problem: Words are sorted lexicographically under an unknown alphabet over lowercase letters. Return any consistent ordering of every letter appearing in the input. Return an empty string for contradictory ordering, cycles, or a longer word before its exact prefix.

Constraints: 1 <= words.length <= 100; 1 <= words[i].length <= 100; words[i] consists of only lowercase English letters.

```
class Solution {
public:
  string alienOrder(vector<string>& words) {
    vector<vector<bool>> g(26, vector<bool>(26));
    vector<bool> s(26);
    int cnt = 0;
    int n = words.size();
    for (int i = 0; i < n - 1; ++i) {
      for (char c : words[i]) {
        if (cnt == 26) break;
        c -= 'a';
        if (!s[c]) {
          ++cnt;
          s[c] = true;
        }
      }
      int m = words[i].size();
      for (int j = 0; j < m; ++j) {
        if (j >= words[i + 1].size()) return "";
        char c1 = words[i][j], c2 = words[i +
          1][j];
        if (c1 == c2) continue;
        if (g[c2 - 'a'][c1 - 'a']) return "";
        g[c1 - 'a'][c2 - 'a'] = true;
        break;
      }
    }
    for (char c : words[n - 1]) {
      if (cnt == 26) break;
      c -= 'a';
      if (!s[c]) {
        ++cnt;
        s[c] = true;
      }
    }
    vector<int> indegree(26);
    for (int i = 0; i < 26; ++i)
      for (int j = 0; j < 26; ++j)
        if (i != j && s[i] && s[j] && g[i][j])
          ++indegree[j];
    queue<int> q;
    for (int i = 0; i < 26; ++i)
      if (s[i] && indegree[i] == 0)
        q.push(i);
    string ans = "";
    while (!q.empty()) {
      int t = q.front();
      ans += (t + 'a');
      q.pop();
      for (int i = 0; i < 26; ++i)
        if (i != t && s[i] && g[t][i])
          if (--indegree[i] == 0)
            q.push(i);
    }
    return ans.size() < cnt ? "" : ans;
  }
};
```

36 LC 310 | Minimum Height Trees (Medium)

Problem: Given an undirected tree on n labeled vertices, return every root that minimizes the rooted tree's height. Output order does not matter.

Constraints: 1 <= n <= 2 * 10^4; edges.length == n - 1; 0 <= a_i, b_i < n; a_i != b_i; All the pairs (a_i, b_i) are distinct.; The given input is guaranteed to be a tree and there will be no repeated edges.

```
class Solution {
public:
  vector<int> findMinHeightTrees(int n,
    vector<vector<int>>& edges) {
    if (n == 1) {
      return {0};
    }
    vector<vector<int>> g(n);
    vector<int> degree(n);
    for (auto& e : edges) {
      int a = e[0], b = e[1];
      g[a].push_back(b);
      g[b].push_back(a);
      ++degree[a];
      ++degree[b];
    }
    queue<int> q;
    for (int i = 0; i < n; ++i) {
      if (degree[i] == 1) {
        q.push(i);
      }
    }
    vector<int> ans;
    while (!q.empty()) {
      ans.clear();
      for (int i = q.size(); i > 0; --i) {
        int a = q.front();
        q.pop();
        ans.push_back(a);
        for (int b : g[a]) {
          if (--degree[b] == 1) {
            q.push(b);
          }
        }
      }
    }
    return ans;
  }
};
```

37 LC 329 | Longest Increasing Path in a Matrix (Hard)

Problem: Given an m x n integers matrix, return the length of the longest increasing path in matrix. From each cell, you can either move in four directions: left, right, up, or down. You may not move diagonally or move outside the boundary (i.e., wrap-around is not allowed).
Constraints: m == matrix.length; n == matrix[i].length; 1 <= m, n <= 200; 0 <= matrix[i][j] <= 2^31 - 1

```
class Solution {
public:
  int longestIncreasingPath(vector<vector<int>>&
    matrix) {
    int m = matrix.size(), n = matrix[0].size();
    constexpr array<int, 5> direction{-1, 0, 1, 0,
      -1};
    vector<vector<int>> degree(m, vector<int>(n));
    queue<pair<int, int>> ready;
    for (int i = 0; i < m; ++i) for (int j = 0; j
      < n; ++j) {
      for (int k = 0; k < 4; ++k) {
        int x = i + direction[k], y = j +
          direction[k + 1];
        if (x >= 0 && x < m && y >= 0 && y < n &&
          matrix[x][y] < matrix[i][j])
          ++degree[i][j];
      }
      if (!degree[i][j]) ready.emplace(i, j);
    }
    int layers = 0;
    while (!ready.empty()) {
      ++layers;
      for (int size = ready.size(); size; --size)
        {
        auto [i, j] = ready.front(); ready.pop();
        for (int k = 0; k < 4; ++k) {
          int x = i + direction[k], y = j +
            direction[k + 1];
          if (x >= 0 && x < m && y >= 0 && y < n
            && matrix[x][y] > matrix[i][j] &&
            --degree[x][y] == 0)
            ready.emplace(x, y);
        }
      }
    }
    return layers;
  }
};
```

38 LC 934 | Shortest Bridge (Medium)

Problem: You are given an n x n binary matrix grid where 1 represents land and 0 represents water. An island is a 4-directionally connected group of 1's not connected to any other 1's. There are exactly two islands in grid. You may change 0's to 1's to connect the two islands to form one island. Return the smallest number of 0's you must flip to connect the two islands.
Constraints: n == grid.length == grid[i].length; 2 <= n <= 100; grid[i][j] is either 0 or 1.; There are exactly two islands in grid.

```
class Solution {
public:
  const static inline vector<int> dirs = {-1, 0,
    1, 0, -1};
  int shortestBridge(vector<vector<int>>& grid) {
    int n = grid.size();
    queue<pair<int, int>> q;
    function<void(int, int)> dfs = [&](int i, int
      j) {
      grid[i][j] = 2;
      q.emplace(i, j);
      for (int k = 0; k < 4; ++k) {
        int x = i + dirs[k], y = j + dirs[k + 1];
        if (x >= 0 && x < n && y >= 0 && y < n &&
          grid[x][y] == 1) {
          dfs(x, y);
        }
      }
    };
    for (int i = 0, x = 1; i < n && x; ++i) {
      for (int j = 0; j < n; ++j) {
        if (grid[i][j]) {
          dfs(i, j);
          x = 0;
          break;
        }
      }
    }
    int ans = 0;
    while (1) {
      for (int h = q.size(); h; --h) {
        auto [i, j] = q.front();
        q.pop();
        for (int k = 0; k < 4; ++k) {
          int x = i + dirs[k], y = j + dirs[k +
            1];
          if (x >= 0 && x < n && y >= 0 && y < n)
            {
            if (grid[x][y] == 1) return ans;
            if (grid[x][y] == 0) {
              grid[x][y] = 2;
              q.emplace(x, y);
            }
          }
        }
      }
      ++ans;
    }
  }
};
```

40 LC 1926 | Nearest Exit from Entrance in Maze (Medium)
Problem: From entrance in a maze, move horizontally/vertically through '.' cells, avoiding '+'. Return the fewest steps to any boundary opening other than the entrance itself, or -1 if no exit is reachable.
Constraints: maze.length == m; maze[i].length == n; 1 <= m, n <= 100; maze[i][j] is either '.' or '+'.; entrance.length == 2; 0 <= entrance_row < m; 0 <= entrance_col < n; entrance will always be an empty cell.

```
class Solution {
public:
  int nearestExit(vector<vector<char>>& maze,
    vector<int>& entrance) {
    int m = maze.size(), n = maze[0].size();
    int dirs[5] = {-1, 0, 1, 0, -1};
    queue<pair<int, int>> q;
    q.emplace(entrance[0], entrance[1]);
    maze[entrance[0]][entrance[1]] = '+';
    for (int ans = 1; !q.empty(); ++ans) {
      for (int k = q.size(); k; --k) {
        auto [i, j] = q.front();
        q.pop();
        for (int d = 0; d < 4; ++d) {
          int x = i + dirs[d], y = j + dirs[d +
            1];
          if (x >= 0 && x < m && y >= 0 && y < n
            && maze[x][y] == '.') {
            if (x == 0 || x == m - 1 || y == 0 ||
              y == n - 1) {
              return ans;
            }
            maze[x][y] = '+';
            q.emplace(x, y);
          }
        }
      }
    }
    return -1;
  }
};
```

47 LC 1615 | Maximal Network Rank (Medium)
Problem: For every pair of cities, their network rank is the number of roads incident to either city, counting a road directly between them once. Return the maximum rank.
Constraints: 2 <= n <= 100; 0 <= roads.length <= n * (n - 1) / 2; roads[i].length == 2; 0 <= a_i, b_i <= n-1; a_i != b_i; Each pair of cities has at most one road connecting them.

```
class Solution {
public:
  int maximalNetworkRank(int n,
    vector<vector<int>>& roads) {
    vector<int> cnt = vector<int>(n, 0);
    vector<vector<int>> g = vector<vector<int>>(n,
      vector<int>(n, 0));
    for (auto& r : roads) {
      int a = r[0], b = r[1];
      g[a][b] = g[b][a] = 1;
      ++cnt[a];
      ++cnt[b];
    }
    int ans = 0;
    for (int a = 0; a < n; ++a) {
      for (int b = a + 1; b < n; ++b) {
        ans = max(ans, cnt[a] + cnt[b] - g[a][b]);
      }
    }
    return ans;
  }
};
```

41 LC 433 | Minimum Genetic Mutation (Medium)
Problem: Genes contain eight characters from A,C,G,T. Return the minimum single-character mutations from startGene to endGene, with each resulting gene present in bank, or -1 if impossible. The starting gene need not be in bank.
Constraints: 0 <= bank.length <= 10; startGene.length == endGene.length == bank[i].length == 8; startGene, endGene, and bank[i] consist of only the characters ['A', 'C', 'G', 'T'].

```
class Solution {
public:
  int minMutation(string startGene, string
    endGene, vector<string>& bank) {
    queue<pair<string, int>> q{{{startGene, 0}}};
    unordered_set<string> vis = {startGene};
    while (!q.empty()) {
      auto [gene, depth] = q.front();
      q.pop();
      if (gene == endGene) {
        return depth;
      }
      for (const string& next : bank) {
        int c = 2;
        for (int k = 0; k < 8 && c; ++k) {
          c -= gene[k] != next[k];
        }
        if (c && !vis.contains(next)) {
          vis.insert(next);
          q.push({next, depth + 1});
        }
      }
    }
    return -1;
  }
};
```

46 LC 1791 | Find Center of Star Graph (Easy)
Problem: The edges form an undirected star graph: one center is joined to every other vertex. Return that center's label.
Constraints: 3 <= n <= 10^5; edges.length == n - 1; edges[i].length == 2; 1 <= u_i, v_i <= n; u_i != v_i; The given edges represent a valid star graph.

```
class Solution {
public:
  int findCenter(vector<vector<int>>& edges) {
    int a = edges[0][0], b = edges[0][1];
    int c = edges[1][0], d = edges[1][1];
    return a == c || a == d ? a : b;
  }
};
```

42 LC 1129 | Shortest Path with Alternating Colors (Medium)

Problem: Edges are directed and colored red or blue. From node 0, return the shortest path length to every node using strictly alternating edge colors; use -1 for unreachable nodes. Parallel edges and self-loops may occur.

Constraints: 1 <= n <= 100; 0 <= redEdges.length, blueEdges.length <= 400; redEdges[i].length == blueEdges[j].length == 2; 0 <= a_i, b_i, u_j, v_j < n

```
class Solution {
public:
  vector<int> shortestAlternatingPaths(int n,
    vector<vector<int>>& redEdges,
    vector<vector<int>>& blueEdges) {
    vector<vector<vector<int>>> g(2,
      vector<vector<int>>(n));
    for (auto& e : redEdges) {
      g[0][e[0]].push_back(e[1]);
    }
    for (auto& e : blueEdges) {
      g[1][e[0]].push_back(e[1]);
    }
    queue<pair<int, int>> q;
    q.emplace(0, 0);
    q.emplace(0, 1);
    vector<vector<char>> vis =
      vector<vector<char>>(n, vector<char>(2, 0));
    vector<int> ans(n, -1);
    int d = 0;
    while (!q.empty()) {
      for (int k = q.size(); k; --k) {
        auto [i, c] = q.front();
        q.pop();
        if (ans[i] == -1) {
          ans[i] = d;
        }
        vis[i][c] = true;
        c ^= 1;
        for (int& j : g[c][i]) {
          if (!vis[j][c]) {
            q.emplace(j, c);
          }
        }
      }
      ++d;
    }
    return ans;
  }
};
```

43 LC 1514 | Path with Maximum Probability (Medium)

Problem: In an undirected graph with independent edge success probabilities, find a path from start to end maximizing the product of its edge probabilities. Return that probability, or 0 if unreachable.

Constraints: 2 <= n <= 10^4; 0 <= start, end < n; start != end; 0 <= a, b < n; a != b; 0 <= succProb.length == edges.length <= 2*10^4; 0 <= succProb[i] <= 1; There is at most one edge between every two nodes.

```
class Solution {
public:
  double maxProbability(int n,
    vector<vector<int>>& edges, vector<double>&
    succProb, int start_node, int end_node) {
    using pdi = pair<double, int>;
    vector<vector<pdi>> g = vector<vector<pdi>>(n,
      vector<pdi>{});
    for (int i = 0; i < edges.size(); ++i) {
      int a = edges[i][0], b = edges[i][1];
      double p = succProb[i];
      g[a].emplace_back(p, b);
      g[b].emplace_back(p, a);
    }
    vector<double> dist(n);
    dist[start_node] = 1;
    priority_queue<pdi> pq;
    pq.emplace(1, start_node);
    while (!pq.empty()) {
      auto [w, a] = pq.top();
      pq.pop();
      if (dist[a] > w) {
        continue;
      }
      for (auto [p, b] : g[a]) {
        auto nw = w * p;
        if (nw > dist[b]) {
          dist[b] = nw;
          pq.emplace(nw, b);
        }
      }
    }
    return dist[end_node];
  }
};
```

44 LC 1361 | Validate Binary Tree Nodes (Medium)

Problem: You have n binary tree nodes numbered from 0 to n - 1 where node i has two children leftChild[i] and rightChild[i], return true if and only if all the given nodes form exactly one valid binary tree. If node i has no left child then leftChild[i] will equal -1, similarly for the right child. Note that the nodes have no values and that we only use the node numbers in this problem.

Constraints: n == leftChild.length == rightChild.length; 1 <= n <= 10^4; -1 <= leftChild[i], rightChild[i] <= n - 1

```
class Solution {
public:
  bool validateBinaryTreeNodes(int n, vector<int>&
    leftChild, vector<int>& rightChild) {
    vector<int> indeg(n, 0);
    for (int c : leftChild) {
      if (c != -1) {
        indeg[c]++;
      }
    }
    for (int c : rightChild) {
      if (c != -1) {
        indeg[c]++;
      }
    }
    int root = -1;
    for (int i = 0; i < n; i++) {
      if (indeg[i] == 0) {
        root = i;
        break;
      }
    }
    if (root == -1) {
      return false;
    }
    queue<int> q;
    unordered_set<int> vis;
    q.push(root);
    vis.insert(root);
    while (!q.empty()) {
      int i = q.front();
      q.pop();
      int j = leftChild[i];
      if (j != -1) {
        if (vis.count(j)) {
          return false;
        }
        vis.insert(j);
        q.push(j);
      }
      j = rightChild[i];
      if (j != -1) {
        if (vis.count(j)) {
          return false;
        }
        vis.insert(j);
        q.push(j);
      }
    }
    return vis.size() == n;
  }
};
```